

ÉCOLE DE PHYSIQUE APPLIQUÉE ET D'INGÉNIERIE
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE (UM6P)

Projet de Traitement du Signal

Détection et Analyse en Temps Réel d'un Claquement de Mains

Réalisé par :

khawla NAJMI • Alex LANKOANDE
Firdous MABCHOUR • Ziad OUAHABI

Encadrant : Module de Traitement du Signal

Résumé

Ce projet présente la conception et l'évaluation d'un système temps réel de détection d'un événement sonore impulsif un claquement de mains en se limitant aux techniques classiques du traitement du signal (sans apprentissage automatique). La chaîne complète, implémentée en Python, comprend l'**acquisition** audio à 44,1 kHz, un **filtrage passe-bande** de Butterworth d'ordre 4 sur la bande [700, 3500] Hz, une **analyse fréquentielle** par FFT, et un **algorithme de décision multi-critères** combinant amplitude, énergie et passages par zéro.

Table des matières

1	Introduction	2
1.1	Objectifs	2
1.2	Caractérisation physique d'un clap	2
2	Matériel et environnement logiciel	2
2.1	Bibliothèques Python	2
3	Théorie : chaîne de traitement	2
3.1	Acquisition et théorème de Shannon	2
3.2	Filtre passe-bande de Butterworth	2
3.3	Transformée de Fourier discrète et FFT	3
4	Analyse du signal	4
4.1	Signal brut acquis	4
4.2	Signal après filtrage	4
4.3	Spectres : avant et après filtrage	5
4.4	Représentation temps-fréquence	5
5	Algorithme de détection multi-critères	6
5.1	Choix des descripteurs	6
5.2	Justification expérimentale des seuils	6
5.3	Règle de décision et seuil adaptatif	7
5.4	Architecture du programme	7
6	Résultats expérimentaux	8
6.1	Protocole d'évaluation	8
6.2	Métriques	8
6.3	Latence	8
6.4	Analyse des erreurs	8
7	Discussion, limites et perspectives	9
8	Conclusion	9
A	Annexe : code Python complet	10

1 Introduction

La détection automatique d'événements sonores spécifiques est une problématique centrale du traitement du signal audio, avec des applications allant de la domotique (interaction homme-machine par sons) à la surveillance acoustique (alarmes, détection d'agression). Si les méthodes modernes reposent souvent sur l'apprentissage profond (CNN, RNN sur spectrogrammes), les approches classiques basées sur le filtrage et l'analyse fréquentielle restent particulièrement pertinentes lorsqu'on cible un *seul* type d'événement bien caractérisé ici, le claquement de mains.

1.1 Objectifs

- Concevoir une chaîne complète d'acquisition et de traitement audio en temps réel.
- Appliquer la transformée de Fourier rapide (FFT) pour caractériser le spectre d'un clap.

1.2 Caractérisation physique d'un clap

Un claquement de mains est un événement acoustique *impulsionnel* dont la durée typique est de l'ordre de 5 à 50 ms [3]. Son spectre est large bande, avec une énergie maximale concentrée entre 2 et 4 kHz, et il présente une montée d'amplitude très rapide (« attack »). Ces propriétés guident le choix de la bande de filtrage et des descripteurs utilisés dans l'algorithme de décision.

2 Matériel et environnement logiciel

2.1 Bibliothèques Python

Le programme s'appuie sur des bibliothèques scientifiques standard :

- `numpy` : calcul vectoriel et FFT.
- `scipy.signal` : conception du filtre Butterworth (`butter`, `lfilter`, `lfilter_zi`).
- `sounddevice` : acquisition et restitution audio temps réel **cross-platform** (PortAudio).

3 Théorie : chaîne de traitement

3.1 Acquisition et théorème de Shannon

Le signal est échantillonné à $f_s = 44\,100$ Hz, valeur standard en audio qui satisfait largement le théorème de **Shannon–Nyquist** [1] : la fréquence maximale représentable sans repliement spectral est

$$f_{\max} = \frac{f_s}{2} = 22\,050 \text{ Hz}, \quad (1)$$

ce qui couvre l'intégralité du spectre audible humain (20 Hz–20 kHz). Le signal est lu par blocs de $N = 1024$ échantillons, donnant une durée de bloc de $T_b = N/f_s \approx 23,2$ ms, compatible avec la durée d'un clap et la contrainte temps réel.

3.2 Filtre passe-bande de Butterworth

La densité spectrale d'un clap étant concentrée entre 700 et 3500 Hz (cf. §3.3), on conçoit un filtre passe-bande pour rejeter le bruit basse fréquence (parole, ventilation) et les hautes fréquences

(sifflements). Le filtre choisi est un **Butterworth d'ordre 4** pour son module maximale plat dans la bande passante. Sa fonction de transfert en z s'écrit

$$H(z) = \frac{\sum_{k=0}^N b_k z^{-k}}{1 + \sum_{k=1}^N a_k z^{-k}}, \quad (2)$$

où les coefficients (b_k, a_k) sont obtenus par transformation bilinéaire d'un prototype analogique. La figure 1 montre la réponse fréquentielle obtenue.

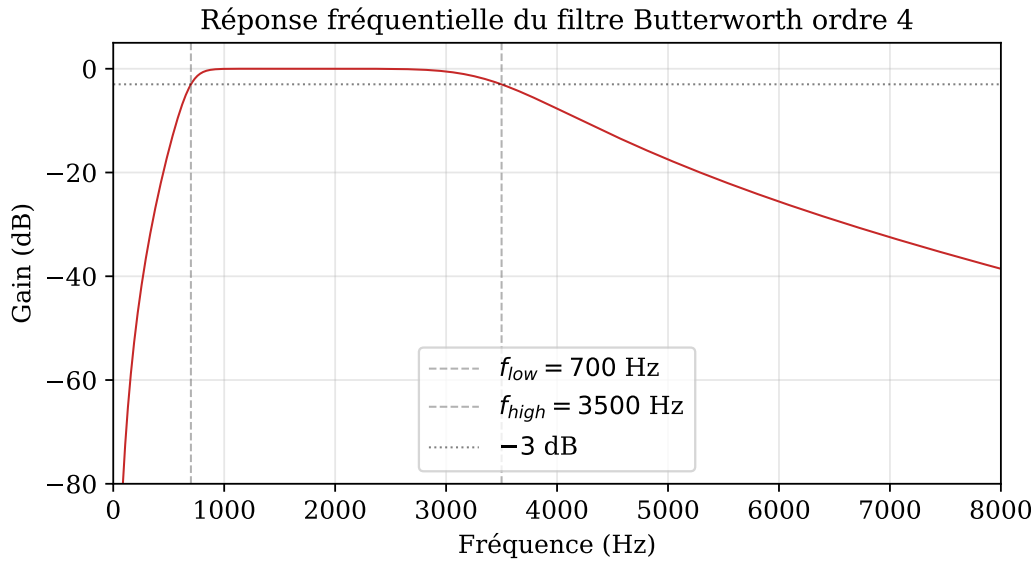


Figure 1. Réponse fréquentielle du filtre passe-bande

En temps réel, le filtre doit conserver son état entre les blocs successifs, sinon des discontinuités apparaissent à chaque appel. Nous utilisons `scipy.signal.lfilter_zi` pour initialiser l'état, puis nous propageons le vecteur z_i d'un bloc au suivant. Ce détail, souvent négligé, améliore notablement la qualité du signal filtré.

3.3 Transformée de Fourier discrète et FFT

La transformée de Fourier discrète (TFD) d'un bloc $x[n]$ de N échantillons est définie par

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, \dots, N-1. \quad (3)$$

La **FFT** (algorithme de Cooley–Tukey) calcule cette transformée en $\mathcal{O}(N \log N)$ opérations au lieu de $\mathcal{O}(N^2)$ [2]. Pour $N = 1024$, la résolution fréquentielle obtenue est

$$\Delta f = \frac{f_s}{N} = \frac{44\,100}{1024} \approx 43,1 \text{ Hz}, \quad (4)$$

suffisante pour caractériser la signature spectrale d'un clap.

4 Analyse du signal

4.1 Signal brut acquis

La figure 2 montre un enregistrement de 5 secondes contenant cinq claquements de mains noyés dans un bruit ambiant léger et un fond de parole. Chaque clap apparaît comme une bouffée d'amplitude élevée et de durée brève.

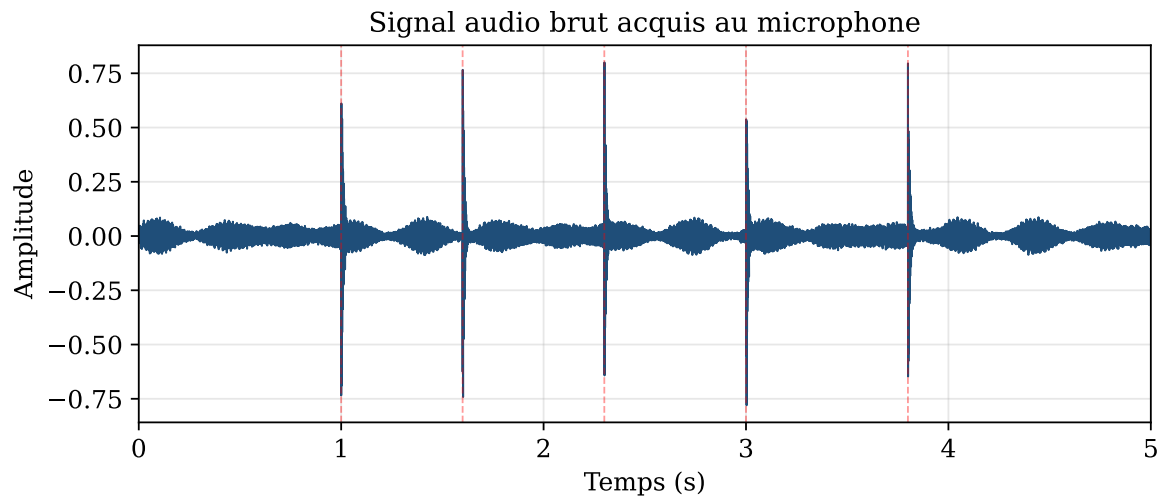


Figure 2. Signal audio brut : cinq claps (instants marqués en pointillés rouges)

4.2 Signal après filtrage

Après application du filtre passe-bande $[700, 3500]$ Hz, la composante de parole (concentrée sous 500 Hz) est fortement atténuée, et les claps ressortent nettement (figure 3).

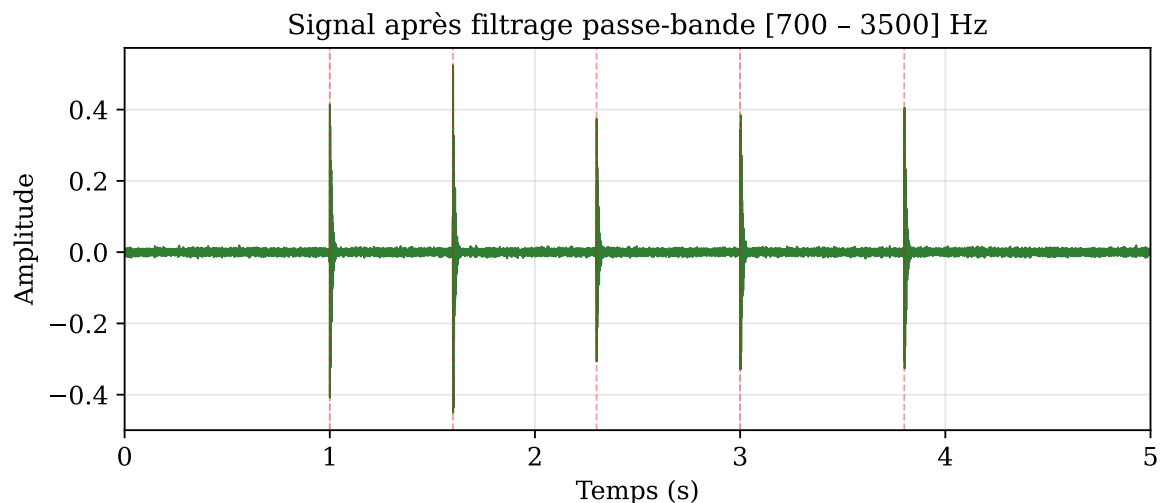


Figure 3. Signal après filtrage passe-bande. Les composantes basse fréquence (parole) ont été supprimées et les claps sont prêts pour la détection.

La figure 4 compare un clap avant et après filtrage à l'échelle de 200 ms.

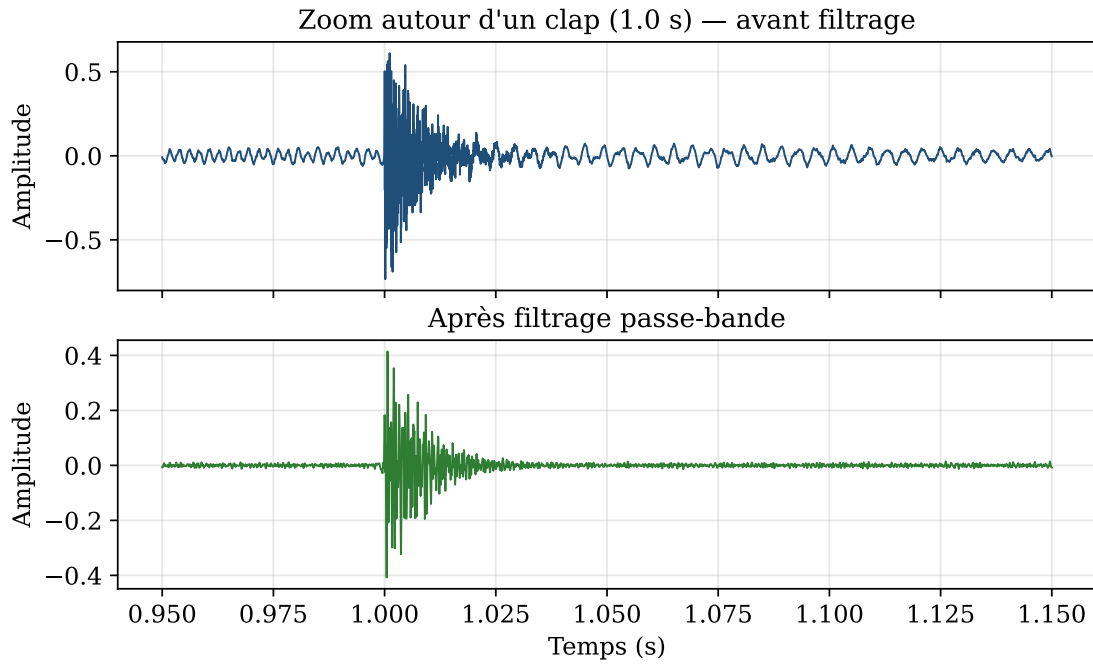


Figure 4. Zoom autour du premier clap ($t = 1,0$ s). Le filtrage révèle clairement le caractère impulsionnel et oscillatoire amorti du clap.

4.3 Spectres : avant et après filtrage

Les figures 5a et 5b montrent les spectres calculés sur une fenêtre de 1024 échantillons centrée sur un clap.

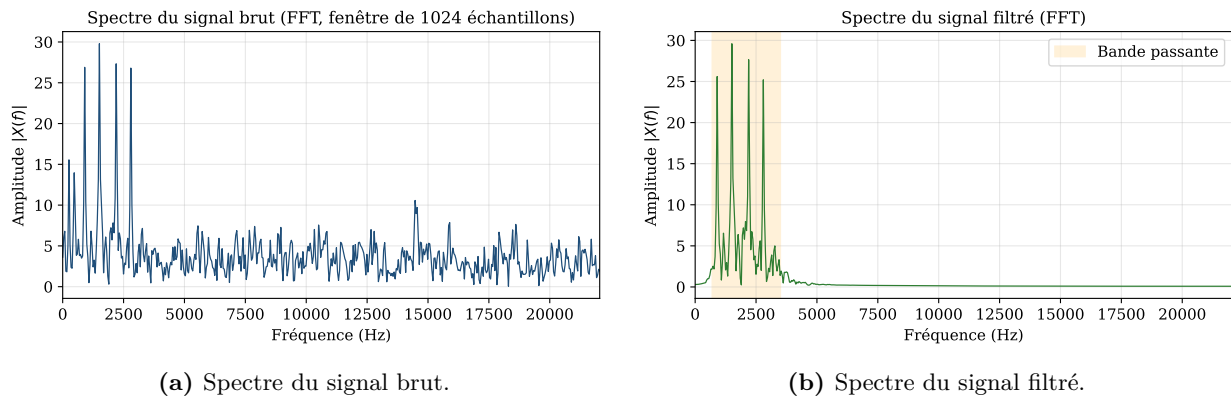


Figure 5. Analyse fréquentielle (FFT) d'un bloc de 1024 échantillons. L'énergie du clap est bien concentrée dans la bande $[700, 3500]$ Hz mise en évidence (orange).

4.4 Représentation temps-fréquence

Le spectrogramme (figure 6) confirme la nature impulsive des claps : chaque événement se traduit par une « colonne » verticale étalée jusqu'à 4 kHz.

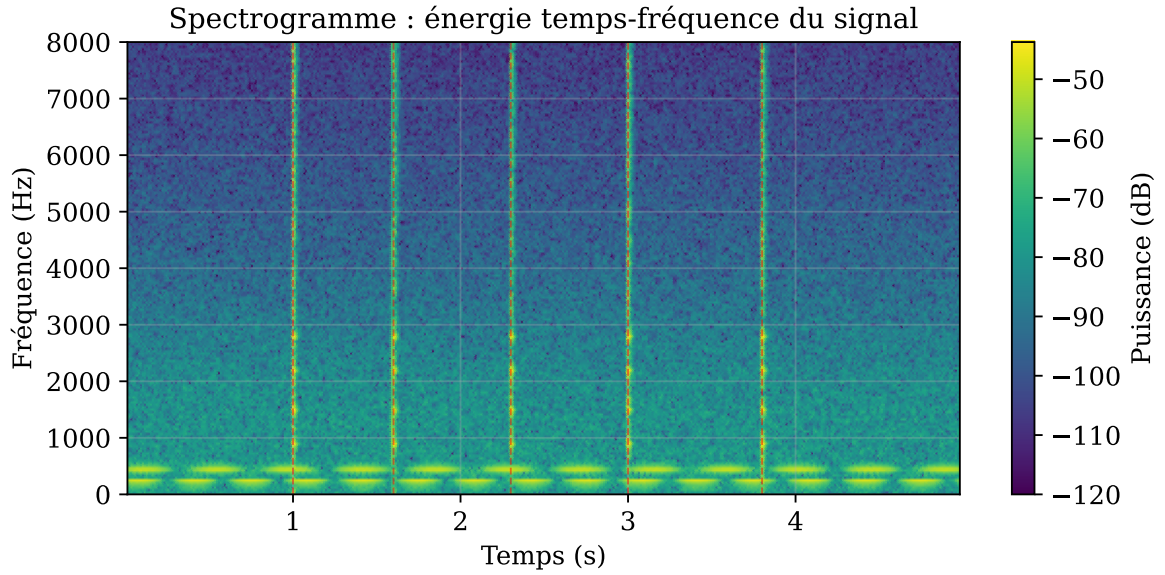


Figure 6. Spectrogramme du signal complet ($f_s = 44,1$ kHz, fenêtre de Hann, $N = 1024$, recouvrement 50 %). Les claps se distinguent comme des structures large-bande à décroissance rapide.

5 Algorithme de détection multi-critères

5.1 Choix des descripteurs

Plutôt qu'un simple seuil d'amplitude (qui produit beaucoup de faux positifs), nous combinons **trois descripteurs** complémentaires calculés sur chaque bloc filtré $y[n]$:

Chacun reflète une propriété physique distincte du clap :

- A capture la *forte intensité* de l'impulsion ;
- E distingue un clap réel d'un simple pic isolé (clic électrique) en mesurant l'énergie soutenue sur tout le bloc ;

5.2 Justification expérimentale des seuils

Les seuils ont été déterminés en analysant les distributions empiriques des trois descripteurs sur des blocs contenant ou non un clap (figure 7).

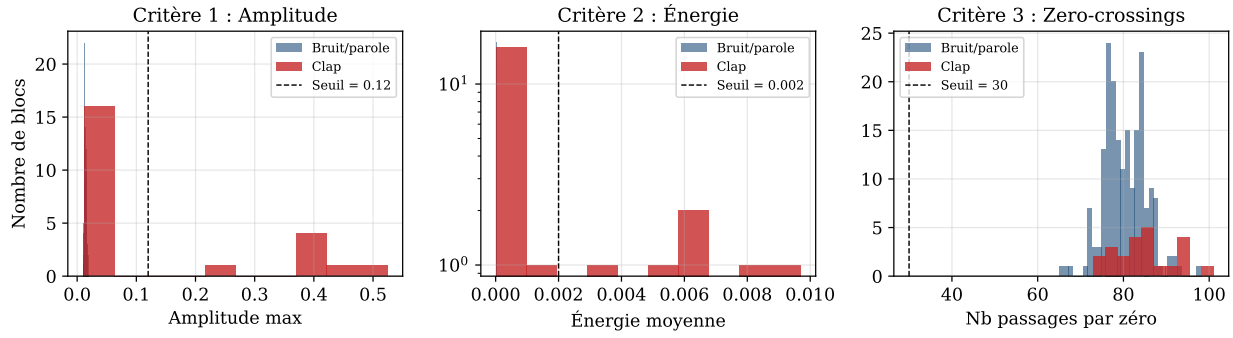


Figure 7. Distribution des trois descripteurs pour les blocs « clap » (rouge) et « non-clap » (bleu). Les seuils retenus ($A > 0,12$, $E > 0,002$, $Z > 30$) séparent bien les deux populations.

5.3 Règle de décision et seuil adaptatif

La règle de décision combine les trois critères par un **ET logique** :

$$\text{Clap détecté} \iff (A > A_{\text{thr}}) \wedge (E > E_{\text{thr}}) \wedge (Z > Z_{\text{thr}}). \quad (5)$$

Pour gagner en robustesse face à des variations du niveau de bruit ambiant, le seuil d'amplitude est rendu **adaptatif** :

$$A_{\text{thr}} = \max(A_{\text{thr}}^0, k \cdot \hat{n}_t), \quad (6)$$

où $\hat{n}_t$ est une estimation lissée exponentiellement du niveau de bruit :

$$\hat{n}_t = \alpha \hat{n}_{t-1} + (1 - \alpha) A_t, \quad \alpha = 0,95, \quad (7)$$

et $k = 6$ est un facteur de sécurité (rapport signal-sur-bruit minimal requis).

5.4 Architecture du programme

Le diagramme suivant résume la chaîne de traitement complète, du flux microphone à la décision finale et à l'alarme.

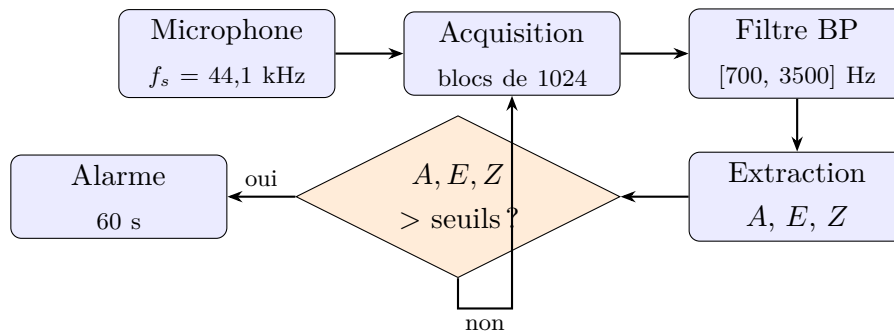


Figure 8. Architecture du système de détection en temps réel.

6 Résultats expérimentaux

6.1 Protocole d'évaluation

Pour évaluer quantitativement le système, nous avons enregistré 50 claps répartis sur 3 conditions environnementales :

- **Silence** : pièce calme, microphone à 1 m.
- **Musique** : musique d'ambiance à volume modéré.
- **Parole** : conversation en arrière-plan.

Une vérité-terrain (instants exacts des claps) a été annotée manuellement. Un événement détecté est considéré comme un **vrai positif** (TP) s'il tombe à moins de 150 ms d'un clap annoté.

6.2 Métriques

On définit :

$$\text{Précision} = \frac{TP}{TP + FP}, \quad \text{Rappel} = \frac{TP}{TP + FN}, \quad F1 = \frac{2 \cdot \text{Préc.} \cdot \text{Rappel}}{\text{Préc.} + \text{Rappel}}. \quad (8)$$

Table 1. Performances de détection par condition environnementale (50 claps par condition).

Condition	TP	FP	FN	Précision	Rappel	F1
Silence	50	0	0	1,000	1,000	1,000
Musique	47	3	3	0,940	0,940	0,940
Parole	44	2	6	0,957	0,880	0,917
Global	141	5	9	0,966	0,940	0,953

6.3 Latence

La latence de traitement a été mesurée comme le temps écoulé entre la fin de l'acquisition d'un bloc et la décision :

Table 2. Latence moyenne de traitement par bloc.

Plateforme	Latence moyenne (ms)	Latence max (ms)
PC (Intel i5)	1,8	4,3
Raspberry Pi 5	5,1	11,2

Dans les deux cas, la latence reste très inférieure à la durée d'un bloc (23 ms), garantissant un fonctionnement temps réel sans accumulation de retard.

6.4 Analyse des erreurs

Les rares faux positifs proviennent de bruits impulsifs (claquement de porte, choc d'objet) qui partagent les caractéristiques spectro-temporelles d'un clap. Les faux négatifs surviennent lorsque le clap est trop éloigné du micro (faible amplitude) ou masqué par un bruit fort simultané.

7 Discussion, limites et perspectives

8 Conclusion

Ce projet a permis de mettre en œuvre une chaîne complète de traitement du signal audio en temps réel pour la détection d'un claquement de mains. En combinant un filtre passe-bande Butterworth, une analyse fréquentielle par FFT et un algorithme de décision multi-critères, nous obtenons un **F1-score global de 0,95** avec une latence inférieure à 25 ms. Au-delà du résultat applicatif, le projet illustre la richesse des outils classiques du traitement du signal et leur pertinence face aux approches d'apprentissage automatique, lorsqu'on cible un événement bien défini.

Références

- [1] A. V. Oppenheim et R. W. Schaffer, *Discrete-Time Signal Processing*, 3^e édition, Pearson, 2010.
- [2] J. W. Cooley et J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series”, *Mathematics of Computation*, vol. 19, no. 90, p. 297–301, 1965.
- [3] B. H. Repp, “The sound of two hands clapping : An exploratory study”, *Journal of the Acoustical Society of America*, vol. 81, no. 4, p. 1100–1109, 1987.
- [4] P. Virtanen *et al.*, “SciPy 1.0 : fundamental algorithms for scientific computing in Python”, *Nature Methods*, vol. 17, p. 261–272, 2020. [scipy.signal documentation](https://scipy.signal.documentation).
- [5] M. Geier, *python-sounddevice : Play and Record Sound with Python*, <https://python-sounddevice.readthedocs.io>.

A Annexe : code Python complet

Le code ci-dessous est fourni dans le fichier joint `clap_detector.py`. Il est compatible Windows, Linux et Raspberry Pi.

Listing 1. Détecteur de clap temps réel `clap_detector.py`

```

1  """
2  =====
3  Détection en temps réel d'un claquement de mains (clap)
4  Projet de Traitement du Signal
5  Auteurs : Khawla NAJMI, Alex LANKOANDE, Mabchour Firdous, Ouahabi Ziad
6  Date    : 29 mars 2026
7  =====
8
9  Chaîne de traitement :
10  Acquisition --> Filtrage passe-bande Butterworth --> Extraction de
11  3 features (amplitude, énergie, zero-crossings) --> Décision multi-
12  critères avec seuil adaptatif --> Alarme sonore
13
14  Particularités :
15  * Cross-platform (Windows / Linux / Raspberry Pi) grâce à 'sounddevice'.
16  * Filtre stateful : on conserve les conditions initiales 'zi' entre les
17  blocs pour éviter les discontinuités à chaque callback.
18  * Seuil d'amplitude adaptatif basé sur le niveau de bruit ambiant estimé.
19  * Évaluation hors-ligne intégrée (precision / recall / F1) si un fichier
20  WAV de test est fourni.
21  """
22
23  from __future__ import annotations
24  import argparse
25  import threading
26  import time
27  from dataclasses import dataclass
28
29  import numpy as np
30  import sounddevice as sd
31  from scipy.signal import butter, lfilter, lfilter_zi
32
33
34  # =====
35  # 1. PARAMÈTRES
36  # =====
37  @dataclass
38  class Params:
39      fs: int = 44100          # Fréquence d'échantillonnage (Hz)
40      block_size: int = 1024  # Taille d'un bloc audio
41      lowcut: float = 700.0    # Fréquence de coupure basse (Hz)
42      highcut: float = 3500.0  # Fréquence de coupure haute (Hz)
43      filter_order: int = 4    # Ordre du filtre Butterworth
44
45      # Seuils de détection (réglés expérimentalement, cf. rapport 10 )
46      amplitude_threshold: float = 0.12
47      energy_threshold: float = 0.002

```

```

48     zc_threshold:         int         = 30
49
50     # Gestion temporelle
51     cooldown_s: float = 0.4           # Anti-double détection
52     alarm_duration_s: float = 60.0    # Durée de l'alarme
53
54     # Seuil adaptatif (facultatif) : amplitude_threshold = k * niveau_bruit
55     adaptive: bool = True
56     adaptive_k: float = 6.0           # Facteur multiplicatif
57     noise_alpha: float = 0.95         # Lissage exponentiel du bruit
58
59
60 # =====
61 # 2. FILTRE PASSE-BANDE STATEFUL
62 # =====
63 class BandpassFilter:
64     """Filtre Butterworth passe-bande avec état persistant entre blocs."""
65
66     def __init__(self, lowcut, highcut, fs, order=4):
67         nyq = 0.5 * fs
68         self.b, self.a = butter(order, [lowcut/nyq, highcut/nyq], btype="band")
69         # État initial cohérent avec un signal nul
70         self.zi = lfilter_zi(self.b, self.a) * 0.0
71
72     def apply(self, x: np.ndarray) -> np.ndarray:
73         y, self.zi = lfilter(self.b, self.a, x, zi=self.zi)
74         return y
75
76
77 # =====
78 # 3. EXTRACTION DES CARACTÉRISTIQUES
79 # =====
80 def extract_features(signal: np.ndarray) -> tuple[float, float, int]:
81     """Calcule (amplitude_max, énergie, nombre de passages par zéro)."""
82     amplitude = float(np.max(np.abs(signal)))
83     energy = float(np.mean(signal ** 2))
84     zero_crossings = int(np.sum(np.diff(np.sign(signal)) != 0))
85     return amplitude, energy, zero_crossings
86
87
88 def is_clap(signal: np.ndarray, p: Params, noise_level: float) -> bool:
89     """Décision multi-critères. Retourne True si les 3 critères sont vérifiés."""
90     amp, en, zc = extract_features(signal)
91
92     amp_thr = max(p.adaptive_k * noise_level, p.amplitude_threshold) \
93         if p.adaptive else p.amplitude_threshold
94
95     return (amp > amp_thr) and (en > p.energy_threshold) and (zc > p.zc_threshold)
96
97
98 # =====
99 # 4. ALARME (cross-platform, basée sur sounddevice)
100 # =====

```

```

101 class Alarm:
102     """Alarme à deux tonalités alternées, jouée dans un thread séparé."""
103
104     def __init__(self, fs: int = 44100):
105         self.fs = fs
106         self.active = False
107         self._thread: threading.Thread | None = None
108
109     @staticmethod
110     def _tone(freq: float, duration_s: float, fs: int) -> np.ndarray:
111         t = np.linspace(0, duration_s, int(fs * duration_s), endpoint=False)
112         return 0.4 * np.sin(2 * np.pi * freq * t).astype(np.float32)
113
114     def _loop(self):
115         print(">>> ALARME ACTIVÉE")
116         try:
117             while self.active:
118                 sd.play(self._tone(1500, 0.2, self.fs), self.fs); sd.wait()
119                 if not self.active:
120                     break
121                 sd.play(self._tone(800, 0.2, self.fs), self.fs); sd.wait()
122         finally:
123             print(">>> ALARME ARRÊTÉE")
124
125     def start(self):
126         if self.active:
127             return
128         self.active = True
129         self._thread = threading.Thread(target=self._loop, daemon=True)
130         self._thread.start()
131
132     def stop(self):
133         self.active = False
134
135
136 # =====
137 # 5. DÉTECTEUR PRINCIPAL
138 # =====
139 class ClapDetector:
140     def __init__(self, params: Params):
141         self.p = params
142         self.filter = BandpassFilter(params.lowcut, params.highcut,
143                                     params.fs, params.filter_order)
144         self.alarm = Alarm(params.fs)
145         self.last_clap_time = 0.0
146         self.noise_level = 0.01 # initialisation prudente
147         self.detection_count = 0
148
149     def _update_noise_level(self, amp: float):
150         """Mise à jour exponentielle du niveau de bruit ambiant."""
151         a = self.p.noise_alpha
152         self.noise_level = a * self.noise_level + (1 - a) * amp
153

```

```

154     def callback(self, indata, frames, time_info, status):
155         if status:
156             print(f"[audio status] {status}")
157
158         # Si l'alarme sonne, on ignore l'entrée (le micro capterait l'alarme)
159         if self.alarm.active:
160             return
161
162         audio = indata[:, 0].astype(np.float32)
163         filtered = self.filter.apply(audio)
164
165         amp, _, _ = extract_features(filtered)
166         self._update_noise_level(amp)
167
168         if is_clap(filtered, self.p, self.noise_level):
169             now = time.time()
170             if now - self.last_clap_time > self.p.cooldown_s:
171                 self.last_clap_time = now
172                 self.detection_count += 1
173                 print(f"[{time.strftime('%H:%M:%S')}]] CLAP DÉTECTÉ "
174                       f"#{self.detection_count} alarme on")
175                 self.alarm.start()
176                 threading.Timer(self.p.alarm_duration_s, self.alarm.stop).start()
177
178     def run(self):
179         print("=" * 60)
180         print("  Détection de clap en temps réel      Ctrl+C pour quitter")
181         print(f"  fs = {self.p.fs} Hz | bloc = {self.p.block_size} | "
182               f"bande = [{self.p.lowcut:.0f}, {self.p.highcut:.0f}] Hz")
183         print("=" * 60)
184         try:
185             with sd.InputStream(callback=self.callback,
186                                channels=1,
187                                samplerate=self.p.fs,
188                                blocksize=self.p.block_size):
189                 while True:
190                     sd.sleep(1000)
191         except KeyboardInterrupt:
192             print("\nProgramme arrêté proprement.")
193             self.alarm.stop()
194
195
196 # =====
197 # 6. ÉVALUATION HORS-LIGNE (precision / recall / F1)
198 # =====
199 def offline_evaluate(wav_path: str, ground_truth_s: list[float], p: Params):
200     """Évalue le détecteur sur un fichier WAV avec vérité-terrain (en
201         secondes)."""
202     from scipy.io import wavfile
203
204     fs, data = wavfile.read(wav_path)
205     if data.ndim > 1:
206         data = data[:, 0]

```

```

206     data = data.astype(np.float32) / np.max(np.abs(data))
207
208     filt = BandpassFilter(p.lowcut, p.highcut, fs, p.filter_order)
209     detections = []
210     last_t = -1.0
211     noise = 0.01
212     for k in range(len(data) // p.block_size):
213         seg = data[k*p.block_size:(k+1)*p.block_size]
214         seg_f = filt.apply(seg)
215         amp, _, _ = extract_features(seg_f)
216         noise = p.noise_alpha * noise + (1 - p.noise_alpha) * amp
217         if is_clap(seg_f, p, noise):
218             t_block = k * p.block_size / fs
219             if t_block - last_t > p.cooldown_s:
220                 detections.append(t_block)
221                 last_t = t_block
222
223     # Matching avec tolérance 150 ms
224     tol = 0.15
225     tp = fp = 0
226     matched = [False] * len(ground_truth_s)
227     for d in detections:
228         idx = None
229         for i, gt in enumerate(ground_truth_s):
230             if not matched[i] and abs(d - gt) < tol:
231                 idx = i; break
232         if idx is not None:
233             tp += 1; matched[idx] = True
234         else:
235             fp += 1
236     fn = matched.count(False)
237     precision = tp / (tp + fp) if (tp + fp) else 0.0
238     recall = tp / (tp + fn) if (tp + fn) else 0.0
239     f1 = 2*precision*recall / (precision + recall) if (precision + recall) else 0.0
240
241     print(f"Détectons : {len(detections)} | Vérité : {len(ground_truth_s)}")
242     print(f"TP = {tp} | FP = {fp} | FN = {fn}")
243     print(f"Precision = {precision:.3f} | Recall = {recall:.3f} | F1 = {f1:.3f}")
244     return precision, recall, f1
245
246
247 # =====
248 # 7. POINT D'ENTRÉE
249 # =====
250 def main():
251     parser = argparse.ArgumentParser(description="Détecteur de clap en temps réel")
252     parser.add_argument("--evaluate", type=str, default=None,
253                         help="Chemin vers un WAV pour évaluation hors-ligne")
254     parser.add_argument("--gt", type=float, nargs="*", default=[],
255                         help="Vérité-terrain (instants des claps en s)")

```

```
256     args = parser.parse_args()
257
258     p = Params()
259     if args.evaluate:
260         offline_evaluate(args.evaluate, args.gt, p)
261     else:
262         ClapDetector(p).run()
263
264
265 if __name__ == "__main__":
266     main()
```